

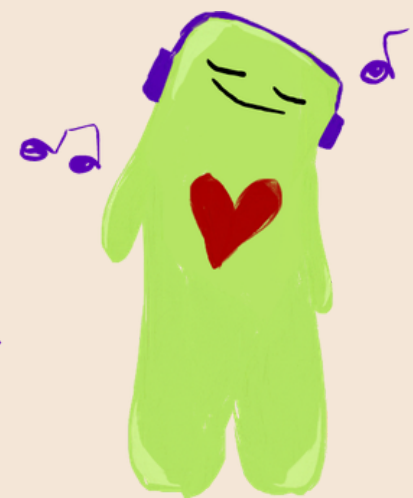


Sorriso do Cerrado

Loja de artesanato



Integrantes



Pedro H.
Cavalcanti



Samantha
Yumi



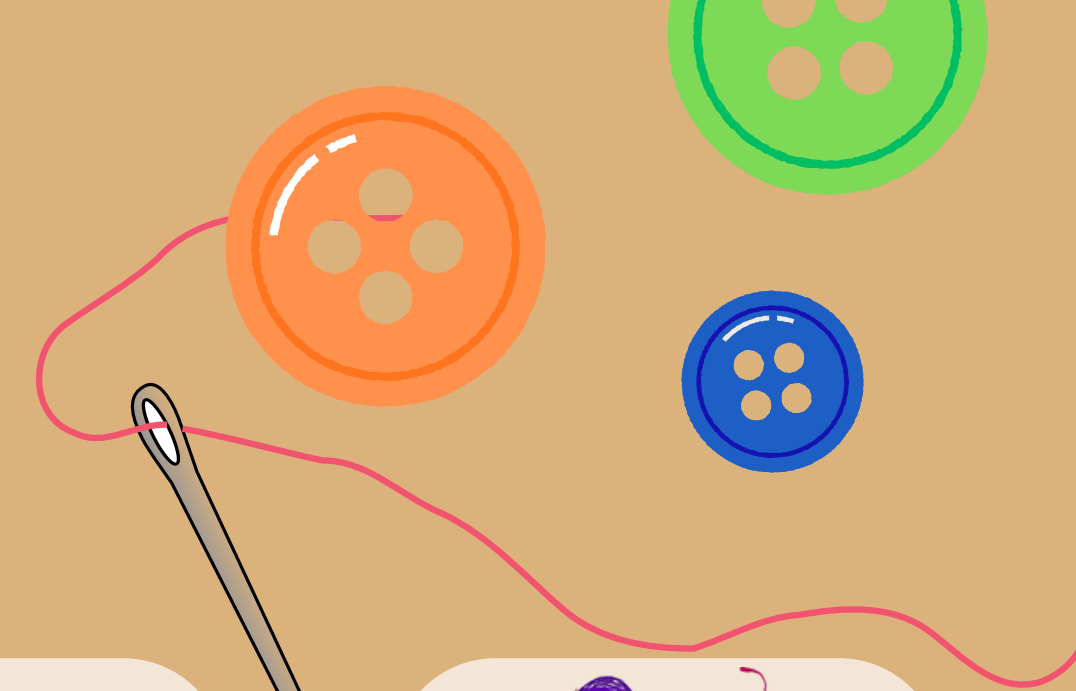
Samuel
Batista



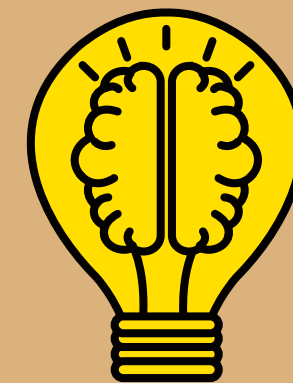
Vinicios
Trindade



Wictor
Emanoel



O projeto



O Sorriso do Cerrado é uma aplicação web desenvolvida na modalidade de “Vitrine Virtual”, estabelecendo uma conexão entre artesãos e clientes interessados, dentro do ambiente web.



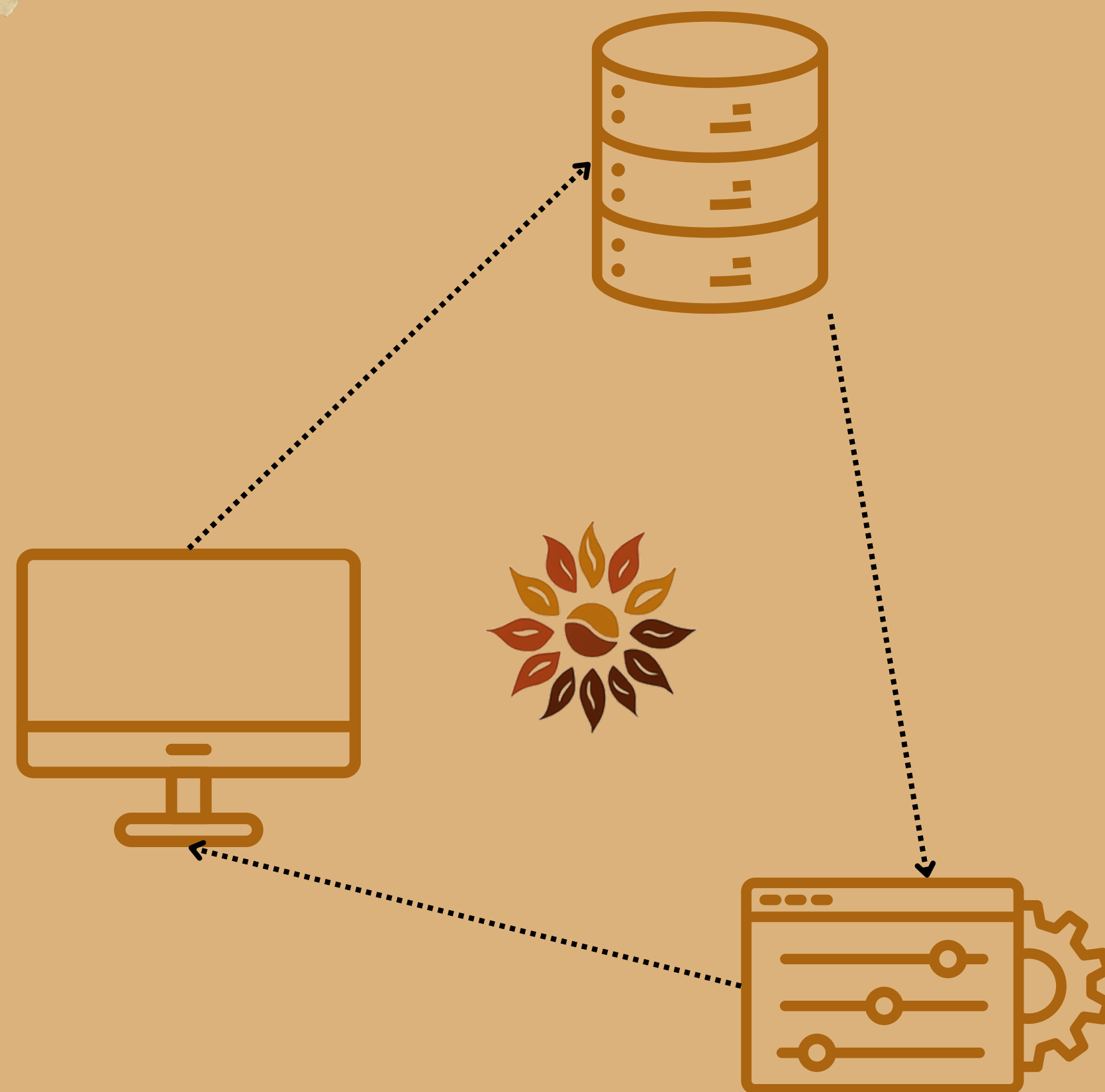
Desenvolvedores de produtos artesanais podem se cadastrar e exibir o seu material dentro da plataforma, enquanto usuários interessados são capazes de adquirir este material e interagir com os provedores dentro do site.



O sistema funciona basicamente através da comunicação entre três diferentes “faces”:

- **Interface de usuários** = Front-End
- **Lógica do sistema** = Back-End
- **Estrutura de dados** = Banco de Dados

Estas três interfaces juntas compõem a aplicação como um todo e mantêm o funcionamento interagindo entre si.



Principais Funcionalidades



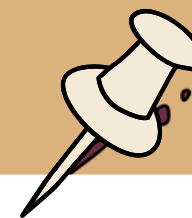
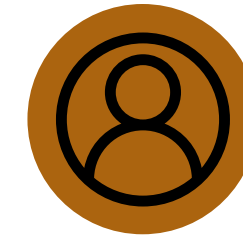
Barra de pesquisa: Procura por produtos do catálogo.



Favoritos: Lista de itens de interesse e gosto do cliente.



Registro de Conta: Usuários e Artesãos têm contas separadas.



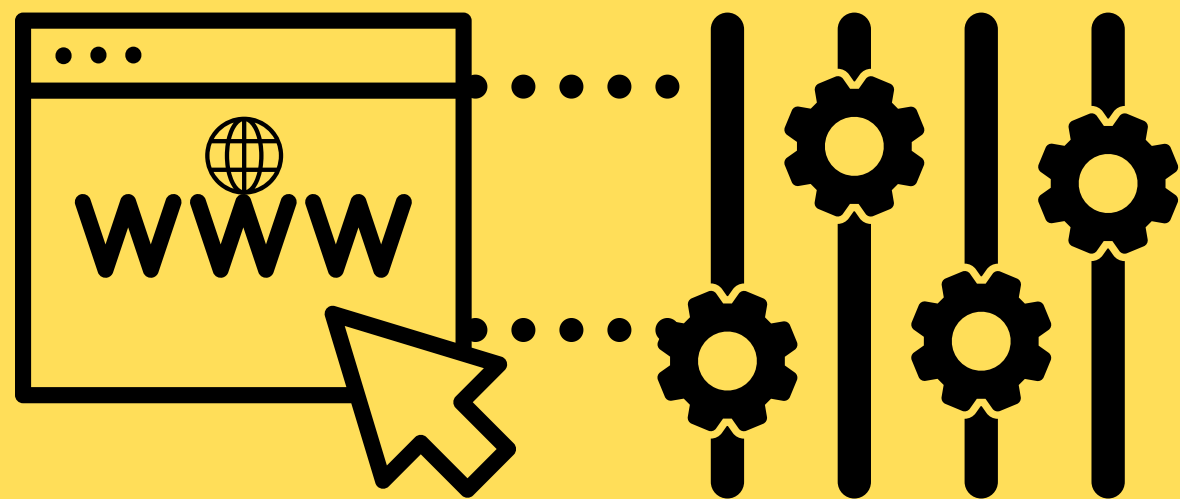
Carrinho: Itens adicionados a lista de compra.



Testes



Todo software de qualidade, seja ele qual for, deve passar por uma série de testes e validações afim de garantir a completude do produto final.



Em aplicações web, como é o caso, é muito importante avaliar aspectos relativos a consistência de lógica, proteção de dados, rotas e a comunicação efetiva com as demais interfaces. A fim de consolidar sua garantia.



Planejamento

Para fins de facilitação a aplicação foi executada localmente, mas utilizou de ferramentas e programas que possibilitaram a simulação de seu funcionamento em ambiente real:



Visual Studio Code: Desenvolvimento dos códigos e aplicações.



My SQL: Construção do banco de dados e armazenamento.



Navegador Web: Execução do programa e sua simulação.

O objetivo dos testes será a verificação do correto funcionamento das funções disponíveis, a proteção de determinadas rotas e permanência de dados.

FERRAMENTA	TIPO DE TESTE	ITENS DE TESTE	TESTES
	Unitário	- Validação de estrutura de e-mail no cadastro de usuário - Validação de preço positivo no produto	- Verificar se o sistema rejeita e-mails inválidos - Verificar se o sistema rejeita preços inválidos ao registrar produtos
	API	- Autenticação e autorização de acesso às rotas protegidas - Atualização de dados do usuário com validação de unicidade	- Validar o controle sobre as diferentes rotas - Validar a correta atualização dos dados de usuário
	End to End	- Produtos favoritos salvos na lista após novo login - Consistência no cálculo e soma do carrinho de compras	- Testar a permanência dos favoritos após login e logout - Verificar o cálculo correto no carrinho de compras

Evidências





Descrição: Verificar se a função interna de validação dos dados do usuário rejeita strings sem os caracteres obrigatórios do email, como o “@”

Passo a passo:

- 1 Iniciar função para validar e-mail
- 2 Enviar string sem “@” ou “.com”
- 3 Executar teste

Critério de Sucesso: A função irá identificar o formato incorreto e não permitirá o avanço da string, impedindo a conclusão do cadastro com dados incorretos de usuários.

Saída Esperada: O valor retornado pela função deverá ser igual a “false”.

CT 001 - Validação de estrutura de e-mail no cadastro de usuário

```
vinicios@vinicios-Aspire-AG15-71P:~/sorrisov4/Sorriso do Cerrado
● d$ npm test > ct001-log.txt
PASS tests/validarEmail.test.js
  ✓ CT001 - deve rejeitar emails inválidos (3 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        0.232 s, estimated 1 s
Ran all test suites.
```

CT 002 - Validação de preço positivo no produto

Unitários

Descrição: Verificar se o sistema impede o cadastro ou atualização de produtos quando o campo de preço recebe valores inválidos, garantindo que apenas valores numéricos maiores que zero sejam aceitos.

Passo a passo:

- 1 Realizar login como artesã ou admin
- 2 Acessar a funcionalidade de cadastro ou atualização de produto
- 3 Informar os dados do produto
- 4 Preencher o campo preço com valor inválido
- 5 Enviar o formulário
- 6 Verificar a resposta do sistema

Critério de Sucesso: O sistema deve impedir a persistência de produtos com preço menor ou igual a zero e permitir a operação apenas quando o valor for estritamente positivo, respeitando também a autenticação de usuários com perfil artesã ou admin.

Saída Esperada: O Sistema deve rejeitar o cadastros ou atualização do produto em todos os cenários, exibindo mensagem de erro informando que o preço deve ser maior que zero. Nenhum produto com valor inválido deve ser salvo ou atualizado no banco de dados



CT 002 - Validação de preço positivo no produto

PASS src/routes/produtos.test.js

CT-002 - Validação de preço positivo no produto

- ✓ Deve aceitar o produto quando o preço for estritamente positivo (4 ms)
- ✓ Deve recusar o produto quando o preço for igual a zero (1 ms)
- ✓ Deve recusar o produto quando o preço for negativo (7 ms)

Test Suites: 1 passed, 1 total

Tests: 3 passed, 3 total

Snapshots: 0 total

Time: 1.05 s, estimated 2 s

Ran all test suites.

❖ PS C:\Users\saman\Desktop\Sorriso-do-Cerrado\Back-End> █



Descrição: Validar se o sistema controla corretamente o acesso às rotas protegidas, garantindo que apenas usuários autenticados e com papel adequado possam executar operações restritas.

Passo a passo:

- 1 Realizar login com usuário de papel cliente
- 2 Copiar o token JWT retornado
- 3 Fazer requisição POST /banners com o token no header Authorization
- 4 Verificar resposta do sistema (acesso negado)
- 5 Realizar login com usuário de papel admin
- 6 Copiar o token JWT do admin
- 7 Fazer requisição POST /banners com token do admin
- 8 Verificar resposta de sucesso da requisição

Critério de Sucesso: O sistema deve bloquear corretamente usuários sem permissão ao tentarem acessar rotas protegidas e permitir apenas usuários com papel admin executarem essas operações

Saída Esperada: sistema deve retornar erro de acesso negado (403) quando a requisição for feita com o token de um usuário com papel cliente, impedindo a execução da operação. Quando a mesma requisição for realizada com o token de um usuário com papel admin, o sistema deve permitir a execução com sucesso, retornando resposta positiva (200 ou 201) e efetuando a criação do banner.

CT 001 - Autenticação e autorização de acesso às rotas protegidas

The screenshot displays a REST client interface with a POST request to `http://localhost:3000/usuarios/login`. The request body is a JSON object containing email and password. The response is a 200 OK status with a JSON body containing a token and user details.

Request:

```
POST http://localhost:3000/usuarios/login
```

Body (JSON):

```
1 {
2   "email": "cliente@email.com",
3   "senha": "123456"
4 }
```

Response:

200 OK • 163 ms • 539 B • 🌐 | ...

Body (JSON):

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwicGFwZSwiOiJjbGllbnR1IiwiaWF0IjoxNzc5MDQ3Nzc5LCJleHAiOjE3Nzk1NzY1Nz19.xQsr4zAby6r0XNtQn2Nec0djLqlby7niWoNGzF6K2oM",
3   "id": 2,
4   "nome": "Cliente Teste",
5   "email": "cliente@email.com",
6   "papel": "cliente"
7 }
```

POST Autenticação e autori: +

No environment

Autenticação e autorização

Save Share

POST

http://localhost:3000/banners

Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

Auth Type

Bearer Token

Token

.....

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body

403 Forbidden · 13 ms · 321 B ·

JSON

Preview

Try with different auth credentials

```
1 {
2   "error": "Acesso negado"
3 }
```

POST Autenticação e autori: +

No environment

Autenticação e autorização

Save Share

POST

http://localhost:3000/usuarios/login

Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

raw

JSON

Schema Beautify

```
1
2 {
3   "email": "admin@email.com",
4   "senha": "admin123"
5 }
```

Body

200 OK · 206 ms · 525 B ·

JSON

Preview

Visualize

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im1hZCI6MTc3OTA4NzAxMSw1ZjoxNzc5MDc1ODExfQ.Y1DUmlo6zvvpNwkoEuqmo1RPYVSzJZHSASryFd1o67s",
3   "id": 1,
4   "nome": "Admin",
5   "email": "admin@email.com",
6   "papel": "admin"
7 }
```

POST Autenticação e autori: +

No environment

Autenticação e autorização

Save

Share

POST

http://localhost:3000/banners

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Tests

Settings

Cookies

Auth Type

Bearer Token

Token

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0=

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

POST Autenticação e autori: +

No environment

Autenticação e autorização

Save

Share

POST

http://localhost:3000/banners

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Tests

Settings

Cookies

form-data

Key	Value	Description	Bulk Edit
✓ Imagem	File banner.jpg		
Key	Text Value	Description	

Body

201 Created + 46 ms + 364 B

JSON

Preview

Visualize

```
1 {
2   "id": 3,
3   "imageUrl": "http://localhost:3000/uploads/1779047592519.jpg"
4 }
```

Descrição: Verificar se o sistema permite que o usuário atualize os seus dados com sucesso. Caso o novo email informado já tenha sido cadastrado, o sistema não deverá concluir a atualização.

Passo a passo:

- 1 Realizar o login na aplicação com o Usuário A
- 2 Clicar em configurações e em seguida em “Editar Dados”
- 3 No campo “Novo email”, digite um e-mail que pertence ao Usuário B, já cadastrado previamente no sistema
- 4 Clique em “Salvar alterações”
- 5 Após a mensagem de erro, corrija o campo “Novo email”, por um endereço válido
- 6 Clique em “Salvar alterações”

Critério de Sucesso: A aplicação protege a unicidade dos dados cadastrados no banco de dados, rejeitando alterações que dupliquem dados e confirmem a alteração quando cumprem todas as regras de validação.

Saída Esperada: Ao tentar salvar as alterações com o email duplicado (Execução A), o sistema deve impedir a atualização. Ao contrário do que deve ocorrer ao salvar as alterações na Execução B, onde o sistema deverá processar a requisição e atualizar o perfil com sucesso.

CT 002 - Atualização de dados do usuário com validação de unicidade

The screenshot shows the Postman interface for a new request. The method is **POST** and the URL is `http://localhost:3000/usuarios/login`. The request body is set to **raw** with **JSON** format, containing the following JSON object:

```
{  "email": "usuario.errado@email.com",  "senha": "123"}
```

The response status is **400 Bad Request**, with a response time of 556 ms and a size of 338 B. The response body is not visible.

The Test Results section shows two tests that passed:

- PASSED** Status code deve ser 400 para credenciais inválidas
- PASSED** Deve retornar 'Usuário ou senha inválidos'

CT 002 - Atualização de dados do usuário com validação de unicidade

Sorriso do Cerrado > New Request

POST http://localhost:3000/usuarios/login

Send

Docs Params Authorization Headers 8 Body **Scripts** Settings Cookies

Pre-request

Post-response

```
1 pm.test("Status code deve ser 400 para credenciais inválidas", function () {
2   |   pm.response.to.have.status(400);
3   | });
4
5 pm.test("Deve retornar 'Usuário ou senha inválidos'", function () {
6   |   var jsonData = pm.response.json();
7   |   pm.expect(jsonData.error).to.eql("Usuário ou senha inválidos");
8   | });
```

Packages </> Snippets

Body Cookies Headers 9 **Test Results 2/2** 400 Bad Request • 556 ms • 338 B • Save Response

Filter Results

PASSED Status code deve ser 400 para credenciais inválidas

PASSED Deve retornar 'Usuário ou senha inválidos'

CT 001 – Produtos favoritos salvos na lista após novo login

E 2 E



Descrição: Verificar se os produtos adicionados aos favoritos permanecem disponíveis após o usuário realizar logout e novo login.

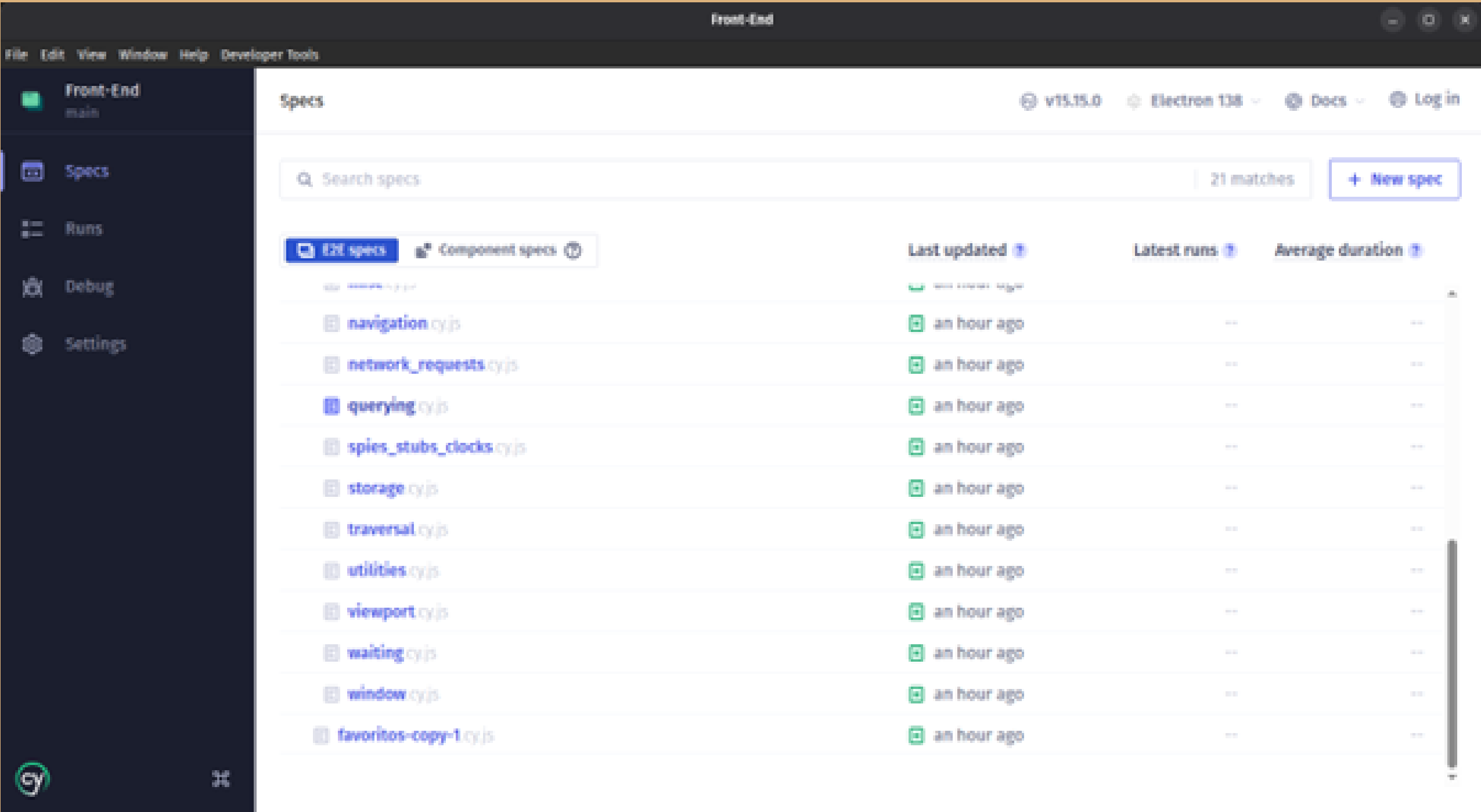
Passo a passo:

- 1 Realizar login no sistema
- 2 Acessar a lista de produtos
- 3 Adicionar um produto à lista de favoritos
- 4 Acessar a página de favoritos e validar presença do item
- 5 Realizar logout
- 6 Realizar novo login com a mesma conta
- 7 Acessar novamente a página de favoritos
- 8 Validar que o produto anteriormente favoritado ainda está presente

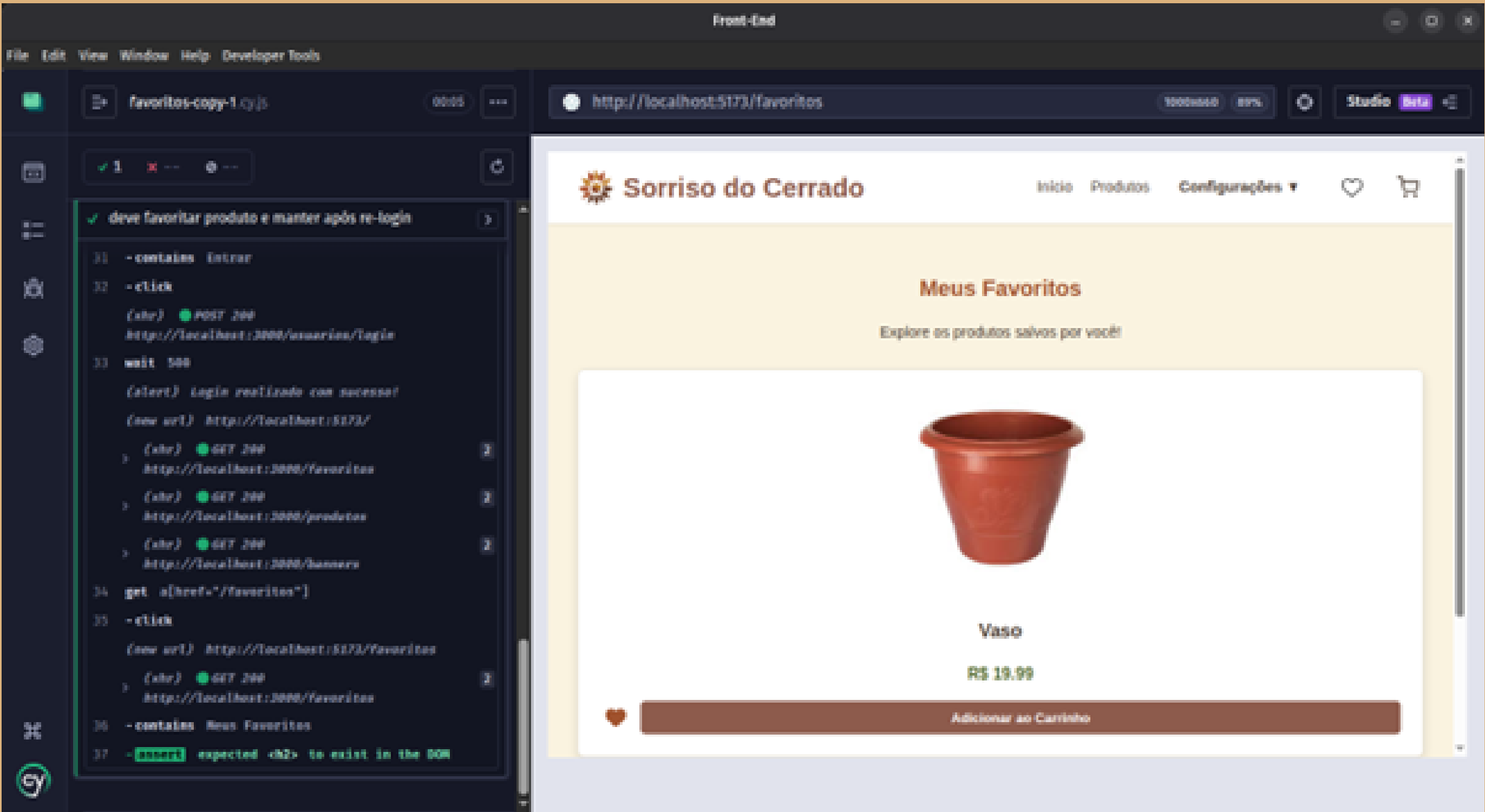
Critério de Sucesso: O sistema deve persistir os dados de favoritos no banco de dados e recuperá-los corretamente após reautenticação do usuário, exibindo os produtos na interface sem perda de informações.

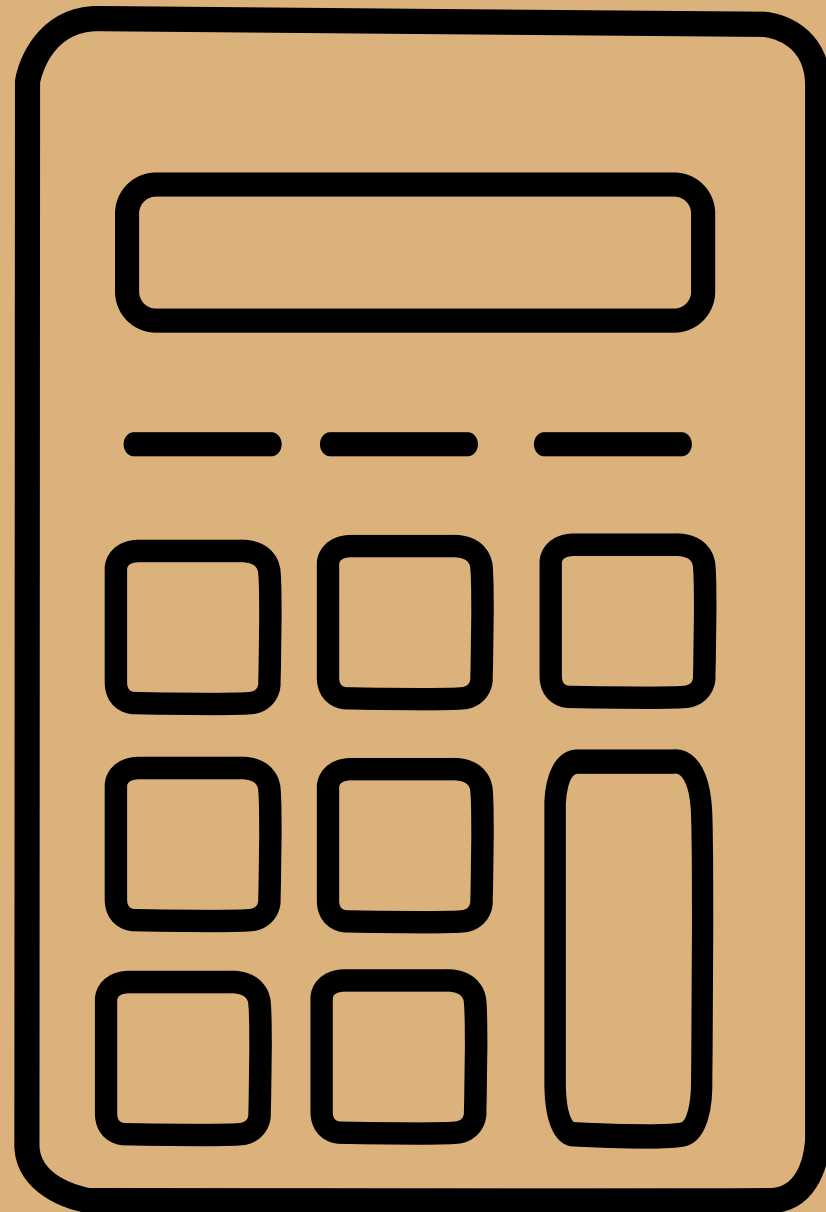
Saída Esperada: A lista de favoritos carregada corretamente com o produto previamente favoritado ainda listado nos favoritos após o novo login

CT 001 - Produtos favoritos salvos na lista após novo login



CT 001 - Produtos favoritos salvos na lista após novo login





Descrição: Verificar se o sistema realiza o cálculo aritmético exato sobre o valor total do carrinho de compras, ao adicionar vários itens com preços e quantidades diferentes, assegurando que o valor final seja correto.

Passo a passo:

- 1 Acessar a vitrine do e-commerce “Sorriso do Cerrado”
- 2 Adicionar o Produto A no carrinho
- 3 Adicionar o Produto B no carrinho duas vezes
- 4 Clicar no carrinho de compras para abrir a tela de pedido
- 5 Validar se o valor total do carrinho corresponde a soma de todos os produtos

Critério de Sucesso: A aplicação executa a lógica matemática de forma correta, atualiza os valores dinamicamente e exhibe, de forma satisfatória, o valor final da compra para o usuário.

Saída Esperada: O sistema deve exibir no valor total do carrinho “R\$245,00”, e não devem ocorrer arredondamentos ou falhar na soma de valores.

CT 002 - Consistência no cálculo e soma do carrinho de compras

The screenshot shows a web browser window with the address bar displaying `http://localhost:5173/carrinho`. The browser's developer tools are open, showing the console on the left and the application on the right.

Terminal Console (Left):

```
1 -> -contains /Vaso de Barro/i
2 closest a
3 -contains /Adicionar/i
4 -click
  (alert) Vaso de Barro foi adicionado ao carrinho!
5 wait 500
6 -contains /Vaso de Barro/i
7 closest a
8 -contains /Adicionar/i
9 -click
  (alert) Vaso de Barro foi adicionado ao carrinho!
10 wait 1000
11 -contains /Bolsa/i
12 closest a
13 -contains /Adicionar/i
14 -click
  (alert) Bolsa foi adicionado ao carrinho!
15 wait 1000
16 visit http://localhost:5173/carrinho
17 wait 2000
18 -contains Total: R$ 500.00
19 a[total] expected attr to be visible
```

Shopping Cart Interface (Right):

Seu Carrinho de Compras

	Vaso de Barro Quantidade: 2 Preço: R\$ 200.00	Remover
	Bolsa Quantidade: 1 Preço: R\$ 100.00	Remover

Total: R\$ 500.00

[Finalizar Compra](#) [Limpar Carrinho](#)

© 2026 Sorriso do Cerrado. Todos os direitos reservados.



Muito Obrigado

Fim da apresentação